DAA LAB 10

Kshitij Kejriwal-252010005

1st April 2026

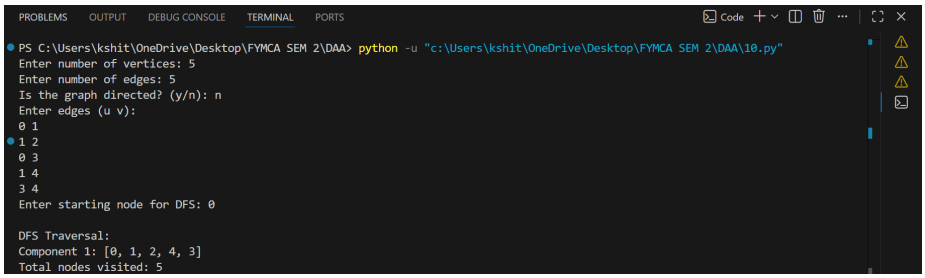
Depth First Search (DFS)

1 Code:

```python
class Graph:
    def __init__(self, vertices, directed=False):
        self.V = vertices
        self.graph = {i: [] for i in range(vertices)}
        self.directed = directed

    def add_edge(self, u, v):
        self.graph[u].append(v)
        if not self.directed:
            self.graph[v].append(u)

    def dfs_util(self, node, visited, traversal):
        visited[node] = True
        traversal.append(node)

        for neighbor in self.graph[node]:
            if not visited[neighbor]:
                self.dfs_util(neighbor, visited, traversal)

    def dfs_full(self, start):
        visited = [False] * self.V
        components = []
        total_nodes = 0

        # DFS from starting node
        if not visited[start]:
            traversal = []
            self.dfs_util(start, visited, traversal)
            components.append(traversal)
            total_nodes += len(traversal)

        # Handle disconnected graph
```

```python
        for i in range(self.V):
            if not visited[i]:
                traversal = []
                self.dfs_util(i, visited, traversal)
                components.append(traversal)
                total_nodes += len(traversal)

        return components, total_nodes


# -------- USER INPUT --------
n = int(input("Enter number of vertices: "))
m = int(input("Enter number of edges: "))

type_graph = input("Is the graph directed? (y/n): ")
directed = True if type_graph.lower() == 'y' else False

g = Graph(n, directed)

print("Enter edges (u v):")
for _ in range(m):
    u, v = map(int, input().split())
    g.add_edge(u, v)

start = int(input("Enter starting node for DFS: "))

# -------- DFS --------
components, total = g.dfs_full(start)

# -------- OUTPUT --------
print("\nDFS Traversal:")

for idx, comp in enumerate(components):
    print(f"Component {idx + 1}: {comp}")

print("Total nodes visited:", total)
```

2 Output:



```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\kshit\OneDrive\Desktop\FYMCA SEM 2\DAA> python -u "c:\Users\kshit\OneDrive\Desktop\FYMCA SEM 2\DAA\10.py"
Enter number of vertices: 5
Enter number of edges: 5
Is the graph directed? (y/n): n
Enter edges (u v):
0 1
1 2
0 3
1 4
3 4
Enter starting node for DFS: 0

DFS Traversal:
Component 1: [0, 1, 2, 4, 3]
Total nodes visited: 5
```

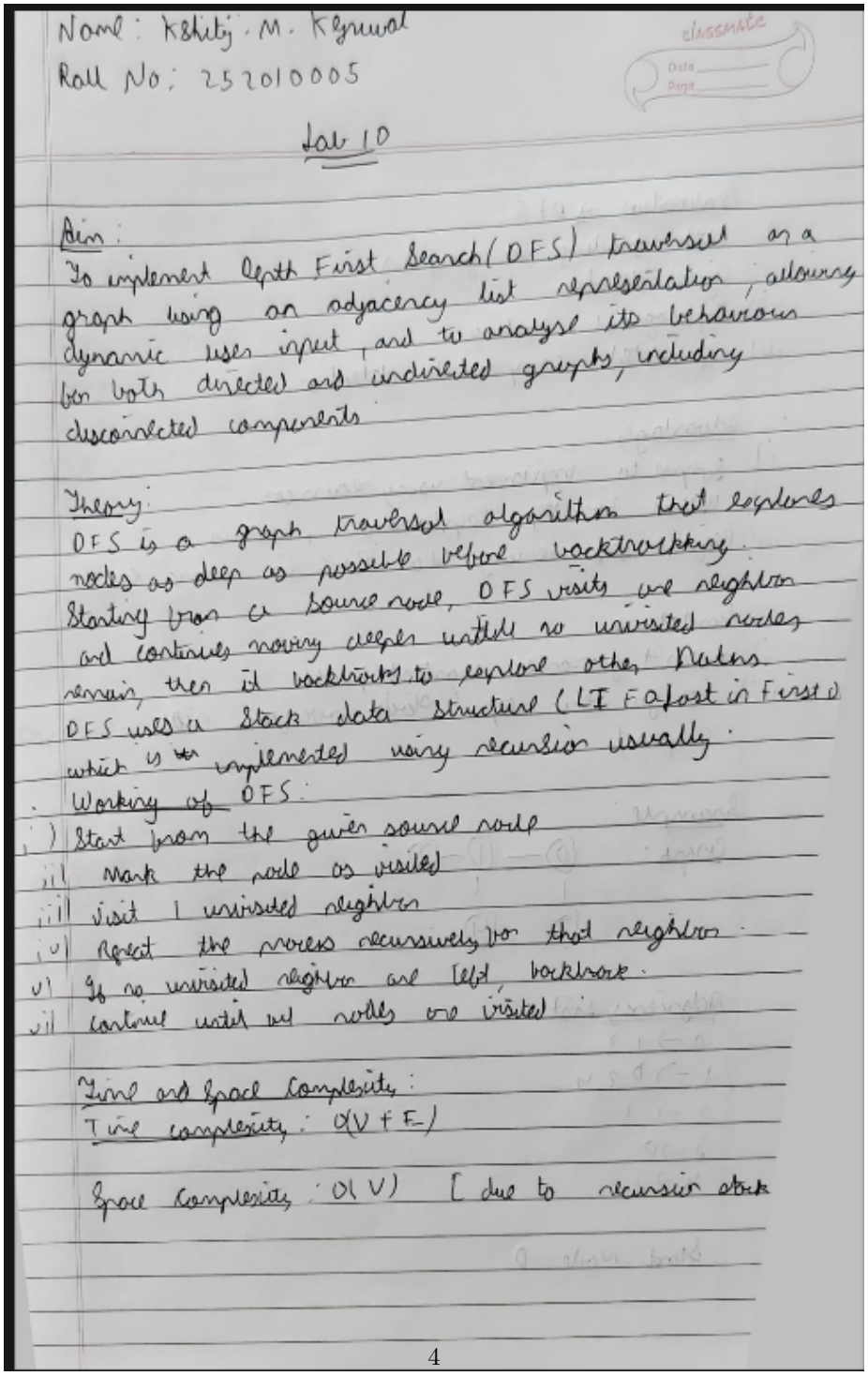
Figure 1: Output

3 Pseudocode:

Algorithm 1 Depth First Search (DFS)

```
procedure DFS(G, node)
    Mark node as visited
    Print node
    for each neighbor of node do
        if neighbor not visited then
            DFS(neighbor)
        end if
    end for
end procedure
```

4 Explanation:



Name: Kshitij M. Kgrwal

Roll No: 252010005

Lab 10

Aim:
To implement Depth First Search (DFS) traversal on a graph using an adjacency list representation, allowing dynamic user input, and to analyse its behaviour for both directed and undirected graphs, including disconnected components.

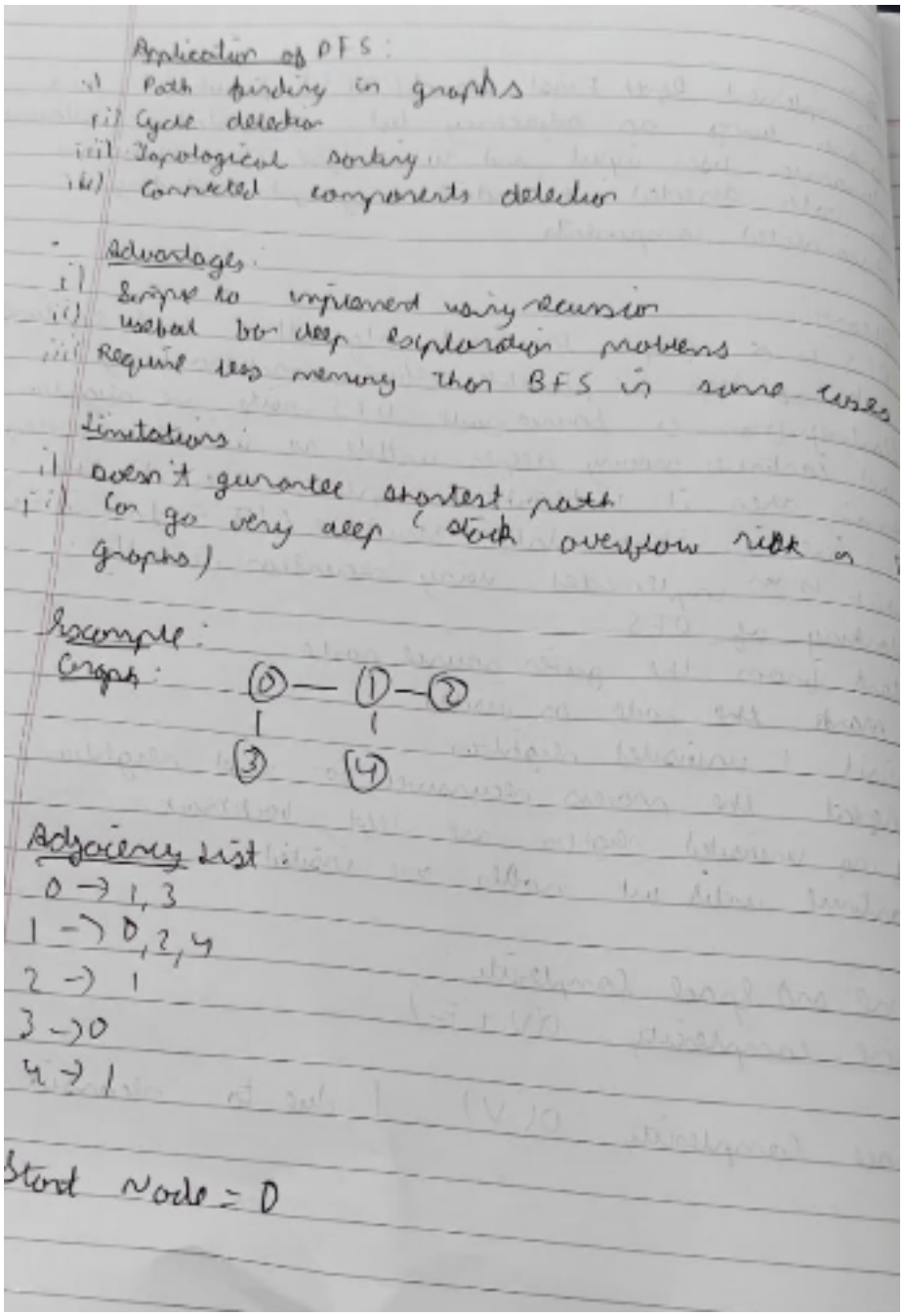
Theory:
DFS is a graph traversal algorithm that explores nodes as deep as possible before backtracking. Starting from a source node, DFS visits one neighbor and continues moving deeper until no unvisited nodes remain, then it backtracks to explore other paths. DFS uses a stack data structure (LIFO Last In First Out) which is implemented using recursion usually.

Working of DFS:
i) Start from the given source node
ii) Mark the node as visited
iii) Visit 1 unvisited neighbor
iv) Repeat the process recursively for that neighbor.
v) If no unvisited neighbor are left, backtrack.
vi) Continue until all nodes are visited

Time and Space Complexity:
Time complexity: O(V + E)

Space Complexity: O(V) [due to recursion stack

Figure 2: Explanation



Application of DFS:

i) Path finding in graphs
ii) Cycle detection
iii) Topological sorting
iv) Connected components detection

Advantages:

i) Simple to implement using recursion
ii) Useful for deep exploration problems
iii) Require less memory than BFS in some cases

Limitations:

i) Doesn't guarantee shortest path
ii) Can go very deep (stack overflow risk in graphs)

Example:

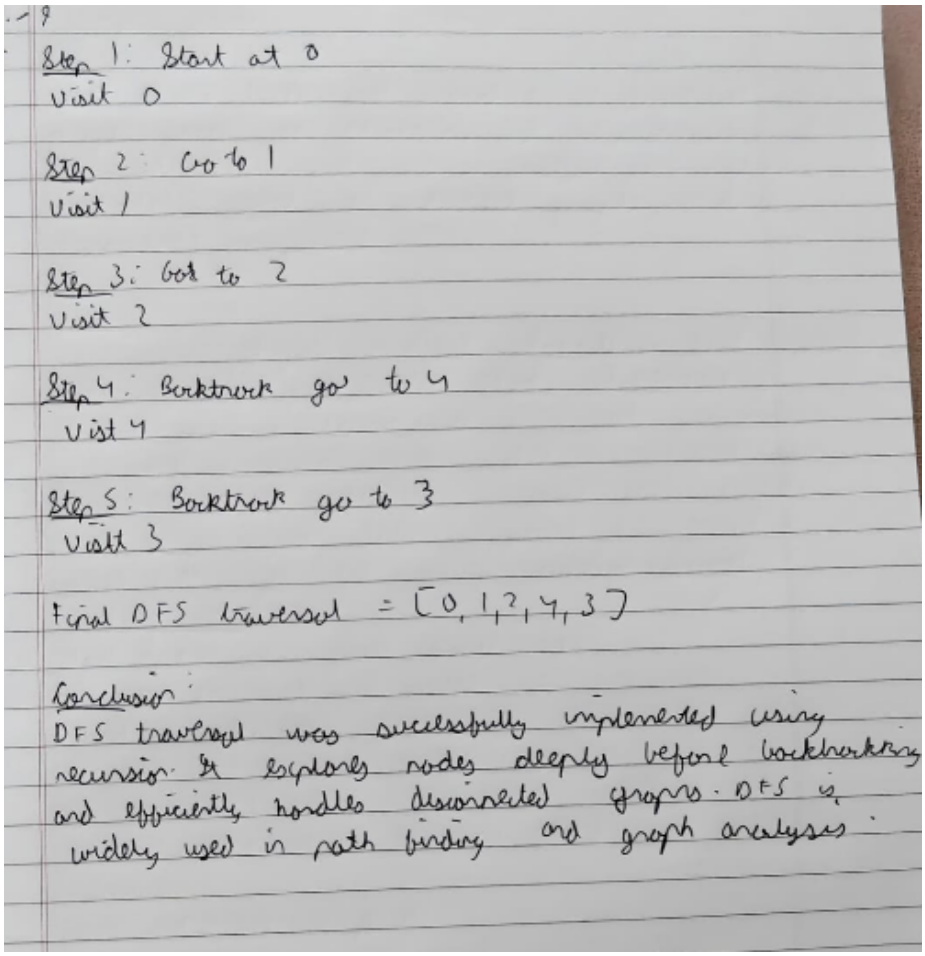
Graph:

Adjacency List

0 → 1, 3
1 → 0, 2, 4
2 → 1
3 → 0
4 → 1

Start Node = 0

Figure 3: Explanation



Step 1: Start at 0
Visit 0

Step 2: Go to 1
Visit 1

Step 3: Got to 2
Visit 2

Step 4: Backtrack go to 4
Vist 4

Step 5: Backtrack go to 3
Visit 3

Final DFS traversal = [0, 1, 2, 4, 3]

Conclusion:
DFS traversal was successfully implemented using recursion. It explores nodes deeply before backtracking and efficiently handles disconnected graphs. DFS is widely used in path finding and graph analysis.

Figure 4: Explanation